

Relaytic-AML: A Local-First, Agent-Assisted Evaluation Lab for Financial-Crime Machine Learning

Tobias Gehra
Independent Researcher
`t.gehra.ai@gmail.com`

July 2026

Abstract

Anti-money laundering (AML) machine-learning experiments are difficult to audit when data residency, temporal validity, graph provenance, agent assistance, review capacity, and public reporting are managed separately. Relaytic-AML is a local-first evaluation lab in which capability-scoped agents and deterministic harnesses convert local runs into provenance-bearing measurements and evidence-bounded release decisions. The architecture is evaluated with temporal PaySim and Elliptic workflows, an external-artifact governance case, and deterministic failure, handoff, and release-gate fixtures. The selected PaySim and Elliptic test PR-AUC point estimates are 0.6388 and 0.6688. Each is reported together with its split, feature, budget, calibration, and test-exposure contract. Across the tested system fixtures, required metric provenance was preserved, raw records were excluded from rowless handoff, and all six injected unsupported-claim cases were blocked. Relaytic-AML contributes an auditable evaluation, governance, and reproducibility architecture for agent-assisted AML experimentation rather than a new detector or detector-superiority result.

1 Introduction

AML modeling is a rare-event, high-stakes evaluation problem. The input may be a stream of mobile-money transfers, card events, account activity, wire messages, customer profiles, or blockchain transactions. Suspicion often appears as a temporal or network pattern: rapid movement of newly received funds, repeated transfers through related accounts, structured amounts, new counterparties receiving many payments, activity inconsistent with a customer profile, or cryptocurrency flows involving higher-risk services and jurisdictions. FATF and FFIEC material describes such pattern-oriented red flags in operational terms [Financial Action Task Force, 2020, Federal Financial Institutions Examination Council, 2014].

Because AML suspicion is often pattern-based, an isolated model metric is easy to misread. A precision-recall area under the curve (PR-AUC) estimate or a precision-at-review-budget estimate becomes interpretable only when the evaluation record shows which data stayed local, which fields were available at decision time, how temporal or graph boundaries were split, whether model selection touched the test partition, and which review capacity was assumed. Precision-recall analysis is particularly informative for highly imbalanced tasks because it focuses on performance for the rare positive class [Saito and Rehmsmeier, 2015]. Agent assistance raises the provenance burden: fluent explanations from large language models (LLMs) or coding agents can drift from the artifact record unless release decisions are tied to machine-checkable evidence.

Relaytic began as a general local-first inference-engineering lab. Relaytic-AML is the domain-specific financial-crime evaluation configuration studied here. It combines capability-scoped

agents, deterministic harnesses, and a local artifact store. Relaytic separates source inspection, experimental design, controlled execution, trace review, and release governance into functional stages backed by deterministic artifact contracts.

Relaytic-AML contributes a local evidence and release-governance layer for AML machine-learning experiments. Benchmark rows exercise the architecture under temporal, graph, operating-point, and reporting pressure. The central question is whether a local evaluation lab can keep evidence, agent assistance, privacy boundaries, and admissible interpretation aligned.

The work is organized around four research questions, each scoped to the workflows and deterministic fixtures evaluated here:

- **RQ1:** Can Relaytic-AML produce reproducible evidence cells for AML-style temporal and graph tasks?
- **RQ2:** Do the implemented gates block the tested leakage paths and unsupported claims?
- **RQ3:** Can the local artifact record support rowless handoff to external agents while preserving provenance?
- **RQ4:** Do the benchmark workflows produce interpretable evidence under explicit split and budget contracts?

The contribution has four linked parts. Evidence cells bind measurements to dataset, split, command, artifact, budget, leakage posture, and operating point. Claim gates govern admissible interpretation separately from the measurement. Rowless handoff exports state, artifact references, and permitted next actions without exporting raw data rows. Deterministic failure and ablation fixtures exercise those boundaries. Local-first software keeps primary data and control on the user’s device while still permitting deliberate collaboration [Kleppmann et al., 2019]. PaySim, Elliptic, and an external-artifact case provide the empirical settings in which these mechanisms are tested.

2 Related Work

The AML benchmark landscape is moving toward larger, more realistic, and more graph-native settings. PaySim remains useful as a synthetic mobile-money fraud simulator for temporal proxy experiments [Lopez-Rojas et al., 2016]. Elliptic introduced a public Bitcoin transaction graph with anonymized node features and temporal labels [Weber et al., 2019]. Elliptic2 and RevTrack/RevClassify shifted attention to suspicious subgraphs and richer blockchain context [Bellei et al., 2024, Song et al., 2024]. Recent preprints such as TransXion, quasi-temporal graph extraction, and BlazingAML, together with published LineMVGNN and continual graph-learning work, treat AML increasingly as a dynamic graph and systems problem [Chen et al., 2026, Poon et al., 2025, Tariq and Hassani, 2026, Ye et al., 2026, Deprez et al., 2025].

Detector papers such as TransXion, BlazingAML, LineMVGNN, and RevClassifyDS focus on detector architecture and benchmark performance. Relaytic-AML addresses the surrounding evaluation process when data is local or licensed, the task may be temporal or graph-based, analyst review capacity matters, and an agent-assisted workflow must still produce auditable records. This places the work near dataset documentation, model reporting, reproducibility practice, experiment tracking, and governance research [Geburu et al., 2021, Mitchell et al., 2019, Pineau et al., 2021, Zaharia et al., 2018].

Experiment-tracking systems preserve runs and artifacts [Zaharia et al., 2018]. Model cards describe trained models, datasheets describe data, reproducibility checklists improve reporting, and agent benchmarks evaluate agent behavior. Relaytic-AML instead connects an executable local measurement to the interpretation that may be released from it. This responsibility matters when evidence comes from licensed files, proxy datasets, temporal graph tasks, or rowless handoff packets rather than from one open leaderboard run.

A closely related line uses LLMs and agents for AML triage, graph-context reasoning, suspicious activity report (SAR) narrative support, compliance serving stacks, and runtime agent governance [Pirmorad, 2025, Naik et al., 2025, 2026, Pervez et al., 2025, Kaptein et al., 2026]. Relaytic-AML instead concentrates on the author-side evaluation path that keeps local AML measurements, external-agent handoff, and public claims aligned.

Relaytic-AML places an executable provenance record and a separate interpretation gate between a detector run and every outward-facing table, handoff packet, or claim. Table 1 locates this responsibility relative to documentation, experiment tracking, detector, and general agent-governance systems.

Table 1: Adjacent systems comparison.

System family	Primary focus	Relaytic-AML scope boundary
Model cards and model reporting [Mitchell et al., 2019]	trained-model reporting	executable AML provenance and interpretation gates
Datasheets and dataset documentation [Geburu et al., 2021]	dataset composition and use	split, leakage, and claim contracts
ML reproducibility checklists [Pineau et al., 2021]	reporting requirements	repository-executed checks and failure fixtures
MLOps experiment tracking [Zaharia et al., 2018]	runs, metrics, and lineage	local AML release-claim admissibility
Agent benchmarks and research-agent evaluations [Chen et al., 2025, Starace et al., 2025]	agent task performance	artifact attachment inside an AML evaluation lab
AML detector and benchmark papers [Weber et al., 2019, Bellei et al., 2024]	detectors, datasets, and benchmark results	evaluation and claim governance around detector runs
AML LLM graph reasoning and triage systems [Pirmorad, 2025, Naik et al., 2026]	AML reasoning and triage	rowless local evidence and release controls
Agentic SAR and compliance narrative assistants [Naik et al., 2025]	investigator-facing narrative support	auditable experimental evidence before downstream writing
Agent governance and runtime trust layers [Pervez et al., 2025, Kaptein et al., 2026]	runtime policy and enforcement	AML measurement, handoff, and claim controls

These system families are complementary. Relaytic-AML integrates source posture, split and leakage policy, budget, operating-point reporting, handoff posture, and claim admissibility for author-side AML evaluation.

Recent agent-evaluation work shows that language-model systems can produce persuasive research artifacts while still failing on reproduction, validation, or expert-level judgment [Chen et al., 2025, Starace et al., 2025, Wijk et al., 2025]. Skill- and tool-using agents expand the set

of actions such systems can perform [Yang et al., 2026]. This broadens the governance surface. Relaytic-AML responds by making model scores, public claims, handoff packets, and paper assets downstream of local artifacts rather than downstream of a conversation transcript.

ResearchLoop represents research questions, task contracts, evidence objects, claim ledgers, and paper bindings as repository-backed state [Xia and Wang, 2026]. FactReview grounds extracted claims in literature and execution-based verification from the reviewer side [Xu et al., 2026]. Safety-gated Model Context Protocol (MCP) work for coding-agent memory separates deterministic control from learned assistance [Iscan, 2026]. Relaytic-AML applies related control principles to author-side AML evaluation, where local data posture, temporal and graph splits, review budgets, rowless handoff, operating points, and release-claim admissibility must remain connected.

3 System Overview

Relaytic-AML is built around one authority rule: authoritative records live in the local workspace rather than in a conversation. Raw data, licensed benchmark files, run summaries, traces, evidence cells, model outputs, tables, figures, and release reports remain on disk. Agents may explain, propose, and repair, but a proposal enters the evidence path only after it is materialized as an inspectable artifact.

The end-to-end control path is shown in Figure 1. Local inputs enter role-scoped execution, each stage writes typed run artifacts, factual cells bind observations to provenance, and release gates determine which interpretations can leave the workspace.

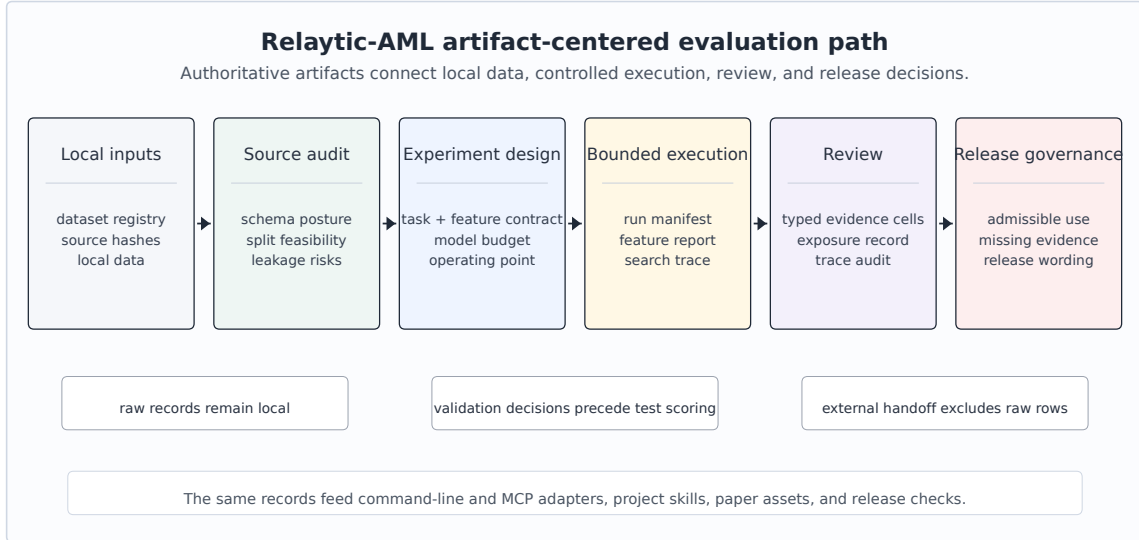


Figure 1: Relaytic-AML local-first architecture: local data and artifacts flow through functional evaluation stages into evidence cells, interpretation gates, and release surfaces.

Figure 1 follows the same state across the functional stages. Dataset registries and split contracts enter source audit and experiment design. Controlled execution writes benchmark manifests, search traces, and feature reports. Review turns observations into typed cells, and release governance combines those cells with audit results before producing reader-facing interpreta-

tions. The same contracts support command-line and MCP adapters, project skill files, external coding-agent integrations, paper assets, and release checks.

3.1 Specialist Roles and State

An agent in Relaytic-AML is a capability-scoped role with declared inputs, allowed tools, output artifacts, and an execution budget. The term does not imply that every role is a separate language-model process. Deterministic specialists implement ingestion, profiling, split checks, training, metric computation, redaction, and release validation. Optional language-model backends are reserved for semantic interpretation and guidance, and their output remains advisory until a deterministic artifact contract accepts it.

The capability profiles divide responsibility without requiring a separate language-model process for each function. Source-audit roles inspect schema, target availability, leakage risks, and split feasibility. Design roles write task, search, feature, and operating-point contracts. Execution roles run approved model paths. Review roles compare candidates under the same contract and reconstruct their traces. Lifecycle and release roles decide whether another controlled attempt is justified, whether a candidate may be promoted, and which wording the artifacts support. Guidance surfaces expose the same recorded state to a user or external agent without becoming a second authority.

3.2 Harness Execution and Control Loops

The shared harness resolves runtime policy before a specialist runs. A capability profile declares artifact read and write scope, raw-row access, semantic access, and external-adapter access. Stage start records the active specialist, input artifacts, source surface, and data-access decisions in an append-only event stream. Tool calls are dispatched through a registry that validates names and argument schemas. Stage completion records output artifacts, emits trace events, runs read-only hooks by default, and writes a checkpoint that can be used for recovery or replay.

Language-model-backed turns use a strict action protocol. Each response must be either a structured `tool_call` with validated arguments or a terminal `respond` action. The loop stops when it reaches a response, a policy block, or a user-confirmation boundary. Turn limits, invalid-action limits, repeated-result detection, and consecutive tool-error limits prevent an agent from drifting into an open-ended conversation or repeatedly invoking a failing tool.

Three control loops operate at different levels. The tool-use loop governs one specialist turn. The evaluation loop moves from baseline construction through bounded challenger branches, validation-only selection, audit, and a completion decision. A current follow-up round may request recalibration, retraining, or an alternate challenger, but branch count and round count remain policy-bounded and the incumbent is retained when promotion criteria are not met. The release loop converts metric artifacts into evidence cells, checks leakage and benchmark role, and either emits admissible wording or records the missing evidence required for a stronger claim. These loops communicate through files and events rather than hidden conversational state.

The external-agent path is rowless by default. Relaytic can export current state, next-action options, artifact shortlists, and safe commands for another model without sending raw transaction rows, secrets, or private local paths. A local LLM can optionally help phrase guidance, but the deterministic guide and evidence artifacts remain the source of truth. The separation between advisory help and local evidence matters for private AML work: outside intelligence may help navigate the run, but data residency and claim provenance stay local unless an operator

deliberately changes policy.

4 Evidence Cell and Claim-Gate Design

A typed evidence cell is the unit that makes an observation auditable. Metric cells record a measured value together with dataset, split, command, artifact field, model budget, leakage posture, calibration, exposure, and operating-point provenance. Invariant cells record a factual system state such as metadata completeness or rowless export, and cannot carry detector metrics. Interpretation is stored in a separate gate output. A cell records what was observed, while the gate determines how that observation may be used.

The factual fields and their separation from interpretation can be seen in Figure 2.

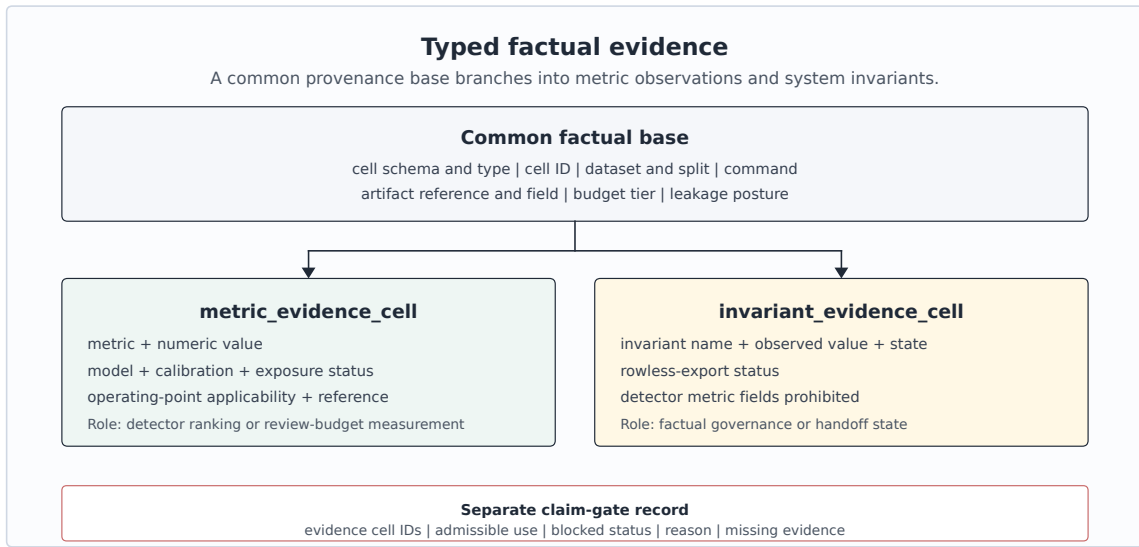


Figure 2: Evidence-cell schema: every reported number carries dataset, split, command, artifact, budget, leakage posture, operating point, metric, and value. Interpretation is stored separately.

Table 2 presents representative evidence cells with compact publication aliases. The underlying evidence records retain the full machine identifiers. Keeping the factual metric record separate from the claim boundary is the central design choice.

Table 2: Representative evidence cells and gate-derived publication roles.

ID	Dataset	Metric	Value	Split	Artifact	Gate-derived use
PS-PR	PaySim	test PR-AUC	0.6388	temporal test	PaySim run, manifest	PaySim temporal-proxy evidence
PS-P@B	PaySim	precision at review budget	0.7033	temporal test	PaySim run, manifest	PaySim temporal-proxy evidence
EL-PR	Elliptic	test PR-AUC	0.6688	graph-time test	graph run, feature table	Elliptic graph-feature evidence
EL-P@B	Elliptic	precision at review budget	1.0000	graph-time test	graph run, feature table	Elliptic graph-feature evidence
E2-PRm	Elliptic2	local repeated context PR-AUC mean	0.9432	pinned artifact TST	E2 run, scorecard	pinned external-artifact context only

Note. The E2 row is pinned external-artifact context only. Upstream cohort equivalence is not established.

At field level, the **PS-PR** alias binds the PaySim test PR-AUC to its dataset, temporal split, command, artifact field, model budget, leakage posture, calibration state, and prior test exposure. The claim gate references that cell identifier and stores admissible use separately. The complete serialized cell and gate examples are provided in the appendix.

Algorithm 1 defines the deterministic construction path from an artifact observation to one of the two factual cell types.

Algorithm 1 Typed evidence-cell creation

- 1: **Input:** observation o , artifact a , dataset registry D , split contract S , run contract R
 - 2: **Output:** validated factual cell c
 - 3: Resolve dataset, split, command, artifact reference, artifact field, budget, and leakage posture from D , S , R , and a .
 - 4: If o is a detector measurement, create a `metric_evidence_cell` with metric, value, model, calibration, exposure, and operating-point provenance.
 - 5: Otherwise create an `invariant_evidence_cell` with invariant name, observed value, invariant state, and rowless-export status.
 - 6: Reject invariant cells that contain detector metric or value fields.
 - 7: Reject metric cells missing any required metric provenance field.
 - 8: Reject interpretive fields such as admissible use or claim strength in either factual cell type.
 - 9: Persist c and pass only its identifier to a separate claim gate.
-

The claim gate is the second half of the design. If the evidence cell is incomplete, a split is leakage-prone, a metric is only a proxy, or a stronger interpretation needs a different dataset or study, the gate preserves the observation and records the additional evidence required. The gate is implemented as a release mechanism, so it changes what the paper artifact pipeline and public surfaces are allowed to say. Algorithm 2 specifies that validation path.

Algorithm 2 Claim-gate validation

- 1: **Input:** proposed claim q , typed factual cells C , benchmark policy P , limitations L
 - 2: **Output:** admissible wording and missing-evidence record
 - 3: Resolve every cell identifier named by q and validate each cell against its declared type schema.
 - 4: Verify source posture, split validity, leakage status, benchmark role, and any applicable operating-point provenance under P .
 - 5: Compare the semantic strength of q with the factual scope supported by C and the limitations in L .
 - 6: If q is supported, emit bounded wording and the exact supporting cell identifiers.
 - 7: Otherwise emit a blocked gate record containing the proposed claim, gate reason, and missing evidence.
 - 8: Permit tables, figures, handoff packets, and release text to consume only the gate output.
-

The resulting routes from current evidence to admissible and stronger future uses are shown in Figure 3.

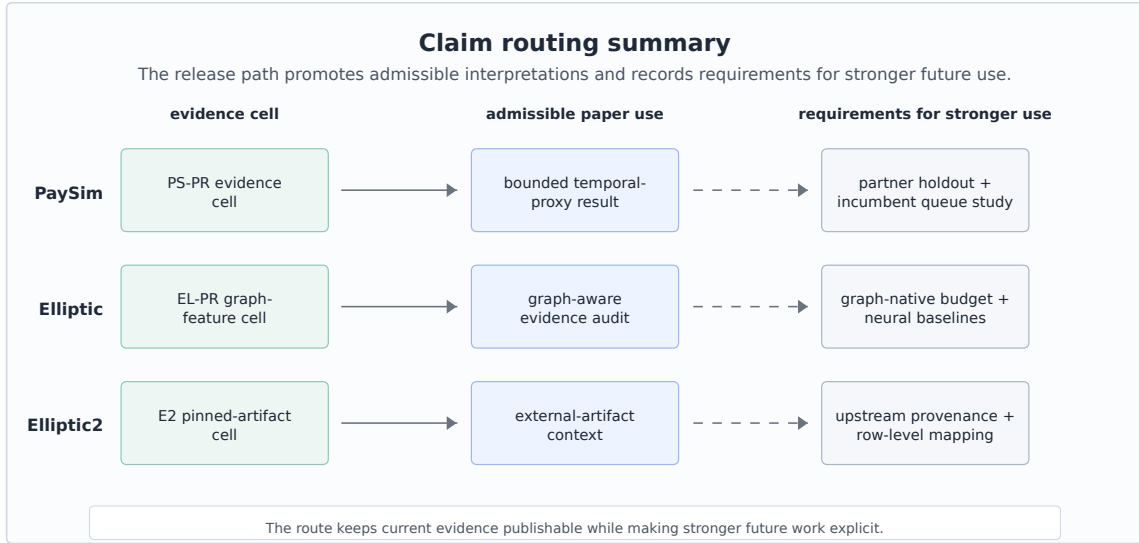


Figure 3: Claim routing summary: current cells map to admissible paper uses and to evidence requirements for stronger future interpretations.

Figure 3 makes the routing behavior concrete. A PaySim row becomes a temporal-proxy demonstration, an Elliptic row becomes graph-feature evidence with temporal provenance, and an Elliptic2 row becomes pinned external-artifact context. The same records specify the requirements for stronger future uses.

5 Experimental Protocol

The experiments test Relaytic-AML as an evaluation lab. The datasets are separated by evidence role. PaySim is the main empirical demonstration because it is fully local and supports a controlled temporal proxy workflow. Elliptic tests temporal graph provenance and feature-view discipline. Elliptic2 tests whether the system can govern a locally held external evaluation artifact without overstating its evidential role.

Dataset scale and exact split boundaries can be seen in Table 3. The corresponding feature, leakage, and metric policies are shown in Table 4. PR-AUC is the primary ranking score because the positive class is rare and analyst capacity is constrained. PaySim uses whole chronological simulator-step boundaries with no gap or embargo. Elliptic uses non-overlapping graph time-step windows. Elliptic2 uses the TRN, VAL, and TST labels supplied by the pinned external RevTrack-format artifact. It is reported separately as non-comparable context.

Relaytic consumes an already constructed, pinned external RevTrack-format artifact named `data_df.pkl`. Relaytic does not construct this artifact. Its upstream construction code and a row-level mapping are unavailable in this repository, and the supplied TRN/VAL/TST partitions define the local evaluation state. The artifact is not claimed to be a reconstruction or an established subset of the currently audited Elliptic2 core. Three documented count states remain distinct: a local audit of the current core records 121,810 subgraphs and 2,763 positives, the RevTrack paper reports 121,810 subgraphs and 2,718 positives [Song et al., 2024], and the pinned artifact contains 110,902 rows and 2,578 positives. The available repository evidence does not explain these count or label differences.

Table 3: Dataset scale and exact split contracts.

Dataset	Unit / positive	Train	Validation	Test
PaySim	transactions / fraud events	1-445: 6,010,937 / 5,007	446-594: 228,103 / 1,552	595-743: 123,580 / 1,654
Elliptic	known-label nodes / illicit nodes	1-29: 26,381 / 2,871	30-39: 8,999 / 1,038	40-49: 11,184 / 636
Elliptic2 context	pinned artifact rows / positives	TRN: 88,738 / 2,054	VAL: 11,059 / 252	TST: 11,105 / 272

Elliptic has 203,769 total nodes. Unknown-label nodes are excluded from fitting and metrics.

Table 4: Feature, leakage, and metric policy.

Dataset	Split policy	Allowed information	Excluded or unavailable information	Primary reporting
PaySim	whole chronological steps, no gap or embargo	row-local amount/type/time, prior-step destination history	mixed balance quartet, raw account IDs, simulator flag	PR-AUC, precision/recall at validation threshold
Elliptic	disjoint time windows, metrics on known labels	source features, same-snapshot structure, combined view	future snapshots, unknown labels as targets	PR-AUC, precision/recall at validation threshold
Elliptic2 context	TRN/VAL/TST labels supplied by pinned external artifact	pinned pooled subgraph summaries	current-core mapping and row-level exclusions unavailable	local repeated PR-AUC, non-comparable context only

Modeling effort is budgeted rather than open-ended. PaySim uses probes on a seeded train-only sample followed by five full-training finalists, including Extra Trees and XGBoost candidates [Geurts et al., 2006, Chen and Guestrin, 2016]. Competitive selection, Platt sigmoid calibration [Platt, 1999], and operating-threshold choice use validation evidence. After protocol freeze, one competitive finalist is evaluated on the fixed test partition. The same test partition had already produced the P4 reference and P6 baseline rows, so it is fixed but not an untouched holdout. Elliptic compares source-provided anonymized features, Relaytic-derived same-step structural features, and their combination. The selected configuration uses LightGBM [Ke et al., 2017] with seed 42. Elliptic2 uses pooled subgraph summaries with LightGBM seeds 11, 42, and 73 as a context workflow. The model-family and search-budget inventory is kept in the appendix.

For PaySim and Elliptic, finalist or feature-view selection used the full declared validation partition. After that selection, the earlier chronological validation subwindow fitted Platt sigmoid calibration. The later subwindow compared calibrated and identity scores by log loss and selected the review threshold. The two nested subwindows are disjoint, but model selection overlaps both because it used the full validation partition. Their exact boundaries and class counts are reported in the appendix validation-subsplit table.

PaySim contains a mixed balance quartet: `oldbalanceOrig` and `oldbalanceDest` describe pre-transaction balances, whereas `newbalanceOrig` and `newbalanceDest` describe post-transaction balances. Relaytic excludes all four conservatively because their availability and simulator consistency do not match the intended pre-decision contract. Raw account identifiers and simulator flags are also excluded as model inputs. Destination history is computed over the full chronological stream as cumulative activity from strictly earlier steps. It carries across train, validation, and test boundaries, but same-step events do not see one another and future steps cannot contribute. The destination identifier is used only as a grouping key, never as a model feature. Amount thresholds are fitted on training data. The isolated contribution of this history family has not been tested and is not claimed. For Elliptic, supervised fitting and metrics use only known labels. Unknown-label nodes may contribute observable features and same-step topology, but never targets or metric rows. The source view retains the dataset’s 94 anonymized local features and 72 supplied one-hop neighbor aggregates as one distinct feature family [Weber et al., 2019]. Elliptic contains no edges between time steps, so those supplied neighborhood aggregates are confined to the dataset snapshot. Their construction is inherited from the source and is not attributed to Relaytic. A second family contains Relaytic-derived structural statistics computed only from edges whose endpoints occur in the same time step. The combined view concatenates the two families. No later snapshot contributes to an earlier prediction.

6 Results

The staged PaySim model-selection path and the evidence visible at each stage can be seen in Table 5.

Table 5: PaySim modeling path.

Stage	Model/contract	Selection evidence	Final test evidence	Role
P4 reference	SGD logistic baseline	source-safe starting point	0.2159	reference row
P6 baseline	Extra Trees baseline	leakage-safe feature set	0.3313	baseline row
Probe screen	best small-sample probe: XGBoost	26 allowed features, probe validation PR-AUC 0.5944	no test evaluation	candidate screening
Full finalist selection	Extra Trees finalist	full-training validation PR-AUC 0.5687, selected without test evidence	not evaluated during selection	model selection
Competitive test	Extra Trees with Platt calibration	one finalist after protocol freeze, P4/P6 exposure disclosed	0.6388	bounded demonstration

PaySim is the most complete local modeling path in the current evidence pack. The P4 reference row had test PR-AUC 0.2159, and the P6 leakage-safe baseline reached 0.3313. The competitive search then used validation evidence only. A small-sample XGBoost probe reached validation PR-AUC 0.5944 on a 750,000-row train-only sample. It is not directly rankable against the five finalists, which were refitted on all 6,010,937 training rows. Among those comparable full-training rows, Extra Trees had the highest validation PR-AUC at 0.5687, 0.0282 above the joint XGBoost and Random Forest runner-up rows at the four-decimal precision shown in the table. No tie-break was required to select the winner. One competitive finalist was evaluated after protocol freeze and reached test PR-AUC 0.6388 and ROC-AUC 0.9683. Raw and calibrated test PR-AUC are both 0.6388, so Platt scaling supports probability and threshold handling rather than a ranking-gain claim. The fixed test partition had prior P4 and P6 exposure and is not presented as untouched.

The PaySim operating point was chosen by taking the score at the requested top 0.5% rank on validation and applying that threshold unchanged to test. Test rows with scores equal to the threshold are included. Ties therefore produced a realized test queue of 1,109 of 123,580 transactions (0.8974%), with precision 0.7033 and recall 0.4716. This queue is more concentrated than the 1.3384% test prevalence, but it still misses more than half of the positive test events. The requested fraction and realized queue must therefore be read separately.

Elliptic is a different evidence contract. The validation-selected source-plus-structural Light-GBM row has validation PR-AUC 0.9767 and later-window test PR-AUC 0.6688. The gap is consistent with temporal shift, validation-specific selection, or both, but the current artifacts do not identify a causal decomposition. The same validation-threshold procedure produced a realized test queue of 36 of 11,184 known-label nodes (0.3219%), with precision 1.0000 and recall 0.0566. The difference from the requested 0.5% follows from applying a fixed threshold with ties rather than forcing a test-set rank. This seed-42 point estimate supports temporal graph provenance and operating-point reporting. It does not isolate a graph-detector advance, because source-provided anonymized features strongly influence the selected view. The numerical thresholds, threshold-selection queues, test queues, calibration choices, and tie policy are collected in the appendix operating-point table. In both workflows, the validation-derived threshold is applied unchanged to test, and scores greater than or equal to the threshold are included. No test-set ranking is used to force an exact 0.5% queue.

Elliptic2 is reported only as a pinned external-artifact context case. On the supplied TST partition, the three-seed local estimate is PR-AUC 0.9432 ± 0.0009 . A separately defined

content-hash partition yields 0.9297. Because the upstream construction and row-level mapping are unavailable and the supplied TST partition had been inspected during an earlier recovery run, the repeated estimate is neither a blind holdout result nor a reproduction or parity claim. No numerical comparison with published RevClassifyDS performance is made because cohort equivalence, upstream table construction, and method reproduction are not established.

Figure 4 separates within-task ranking estimates from validation-threshold review queues. The panels use distinct task contracts and must not be read as a cross-dataset leaderboard.



Figure 4: Evaluation evidence by task contract: local ranking estimates and validation-threshold review queues are shown in separate panels.

In Figure 4, PR-AUC summarizes ranking within a dataset. Precision and recall describe the test rows selected by a validation-derived threshold, while the realized fractions show how ties and score distributions changed queue size.

7 Deterministic Artifact and Release-Gate Evaluation

The system claim is evaluated through deterministic reader and agent tasks. These checks ask whether a reviewer can navigate from the README to the paper evidence, trace PaySim metric provenance, compare baseline and competitive budgets, keep Elliptic2 as context rather than a contribution, export state without raw rows, recover an interrupted run, and block unsupported public claims. Table 6 reports the synthesis. Detailed failure cases, ablations, the invariant map, the hosted-score example, and handoff rows are preserved in the appendix and generated evidence artifacts.

Table 6: Deterministic artifact and release-gate checks.

Check	Failure condition	Mechanism	Observed result	Scope
Metric provenance	A reported number cannot be traced to source, split, command, or artifact.	Required evidence-cell fields and evidence-cell audit.	17/17 required metric fields present, evidence/gate separation audit passed.	Demonstrates traceability on tested paths, not detector optimality.
Budget comparability	Baseline and competitive rows are compared under different contracts.	Dataset, split doctrine, metric, and budget checks.	PaySim PR-AUC changed from 0.3313 to 0.6388 under the same dataset, temporal split, and metric. The competitive route uses a distinct audited feature contract and a larger modeling budget.	Supports comparison within the declared PaySim contracts.
Leakage and selection firewall	Post-event fields or test evidence influence competitive selection.	Feature policy, validation-only selection, and exposure record.	4 balance fields excluded, competitive selection used no test evidence, one competitive finalist evaluated after protocol freeze, prior P4/P6 exposure recorded.	Fixed partition, not an untouched holdout.
Claim-strength gating	Proxy or context rows become unsupported performance or deployment claims.	Public wording lint, publishability matrix, and injected claim cases.	All 6/6 injected unsupported-claim cases were blocked.	Deterministic fixture behavior, not general semantic correctness.
Rowless handoff	An external agent receives raw rows, credentials, or private paths.	Context-export redaction and handoff evaluator.	Rowless handoff preserved next action and allowed tools.	Deterministic fixture result, not a privacy certification.
Interrupted recovery	A user or agent cannot recover current state without artifact literacy.	No-lost-user guide and recovery artifact shortlist.	Recovery guide, partial-run state, and artifact shortlist were emitted.	Deterministic recovery check, not a human study.
Hosted-score wrapper	A third-party score file is mistaken for Relaytic detector novelty.	Schema/hash adapter, evidence cell, redaction report, and claim map.	11 exported fields, 16 blocked fields, no raw rows exported	Hosted detector-output governance only.

Across the tested fixtures, a number with missing provenance, a prohibited feature path, a test-selected finalist, an unsafe handoff packet, or an unsupported claim produces a blocked record instead of reader-facing text. All six injected unsupported-claim cases were blocked. These are deterministic infrastructure checks, distinct from empirical detector performance. They do not measure human usability, provide privacy assurance, or establish production AML validity.

The hosted external-score fixture shows the intended integration point for stronger third-party detectors. A rowless detector-score artifact enters Relaytic with schema and content hashes, not raw rows. Relaytic emits one invariant cell for metadata completeness, redacts unsafe handoff fields, and routes the result as hosted detector-output governance evidence. Future detector outputs can therefore pass through the same local release boundary without being mistaken for a new detector contribution.

8 Limitations and Threats to Validity

PaySim supports a controlled synthetic temporal experiment rather than real-bank AML inference. The fixed test partition had prior P4 and P6 exposure, although competitive model selection remained validation-only and one finalist was evaluated after protocol freeze. Model selection used the full validation partition, which contains the later calibration and threshold-selection subwindows. This overlap may make validation-based selection evidence optimistic without exposing test labels. The isolated contribution of destination history was not tested. A stronger protocol would reserve disjoint chronological surfaces for selection, calibration, threshold choice, and final evaluation.

Elliptic provides a temporal public-blockchain graph task with anonymized and source-supplied features. Unknown labels, inherited neighbor aggregates, and public-chain behavior limit operational interpretation, and the source feature pipeline is not reconstructed independently. The PaySim and Elliptic detector results are single-seed point estimates. Prediction-level scores are absent from the committed aggregate evidence, so confidence intervals cannot be reconstructed faithfully.

The pinned Elliptic2 artifact permits a transparent external-artifact governance case, but its upstream construction and row-level mapping are unavailable. The repository can verify and evaluate the supplied artifact but cannot reconstruct it from the original Elliptic2 data. Its result therefore remains external-artifact context rather than cohort-equivalent or reproduced evidence.

The deterministic fixtures test artifact completeness, redaction, gate decisions, handoff, and recovery behavior. They are not a human usability study, privacy certification, security assessment, or production validation. Human and institutional studies are needed to measure analyst time, investigation quality, organizational adoption, and operational outcomes.

The local-first design reduces external transfer of raw records and strengthens local provenance control, but it does not by itself provide a privacy or security guarantee. Independent raw-data reproduction still depends on lawful access to PaySim, Elliptic, and the pinned external artifact. Repeated runs, partner-approved holdouts, same-queue incumbent comparisons, and graph-native model families remain future empirical work.

9 Reproducibility

The public repository contains the Relaytic framework, the AML case-study implementation, and the manuscript build pipeline. The release bundle records the exact source commit and tag used for the arXiv submission. A clean clone can rebuild the paper and rerun repository-local fixtures, while benchmark reruns require the locally obtained datasets and dependencies listed below.

Repository: <https://github.com/ML-Enthusiast-de/Relaytic>. The immutable release process records the full source revision in the release manifest and arXiv source bundle.

Table 7 separates what a clean clone can reproduce immediately from what requires local benchmark access. The README contains the full regeneration script, while the paper keeps the main path short enough to try without reading the generated audit files first.

Table 7: Reproduction modes and dependencies.

Mode	Command fragment	Output class	Requirement
Paper build	paper-release, paper-arxiv-source	Markdown, LaTeX, bibliography, and vector figures	clean clone, TeX required only for PDF compilation
Source validation	paper-final-preflight	citations, logs, fonts, links, metadata, and release gates	compiled PDF and local TeX tools
Deterministic fixtures	paper-invariants	provenance, claim, handoff, and recovery cases	repo-local fixtures, no benchmark data
Artifact verification	paper-release-integrity	metric/split agreement and evidence authority	committed rowless reports, no retraining
PaySim raw-data rerun	paysim-competitive -budget-tier competitive -run-optional -require-full-rerun	competitive model and operating-point artifacts	local PaySim CSV, sha256 prefix 16910f90577b
Elliptic raw-data rerun	graph-baselines -budget-tier competitive -run-optional -require-full-rerun	graph-feature and operating-point artifacts	local Elliptic bundle, sha256 prefix 93e2e7b2405c plus 2 files
Elliptic2 context rerun	elliptic2-competitive -budget-tier competitive -run-suite -require-full-rerun	pinned-artifact context records	local pinned data_df.pkl and companion RevTrack files, hash verified, prior test exposure disclosed

The first four rows in Table 7 are available from a clean clone. Artifact verification checks committed evidence without retraining. Deterministic fixtures execute repo-local synthetic cases, and the paper build regenerates publication assets. Raw-data benchmark reruns require local PaySim, Elliptic, or Elliptic2/RevTrack data. A zero process exit from an optional command is not by itself evidence that every benchmark branch ran. Each command emits a machine-readable execution status. Adding `--require-full-rerun` makes the command fail when data, dependencies, or requested branches were skipped.

Minimal public check:

```
python -m pip install -e ".[full]"
python -m relaytic.ui.cli release-safety paper-invariants --format json
python -m relaytic.ui.cli release-safety paper-release --format json
python -m relaytic.ui.cli release-safety paper-release-integrity --candidate --format json
python -m relaytic.ui.cli release-safety paper-arxiv-source --format json
python -m relaytic.ui.cli release-safety paper-final-preflight --format json
```

Raw benchmark data is not committed. PaySim and Elliptic require local downloads and are referenced through registry artifacts, split reports, hashes, and command ledgers. A full Elliptic2 context run requires the locally held pinned `data_df.pkl` with SHA-256 `2baa712b67382aeade8d5e72dd07ddbfb1029b359a048c80a2300a3e3abc220` and its companion RevTrack files. A clean clone can verify committed rowless records but cannot construct the pinned artifact from the original Elliptic2 data. When the artifact is available, Relaytic verifies its hash, recounts its supplied splits, executes the local context workflow, and applies the claim-governance checks.

Full benchmark regeneration requires the local datasets named in the README and the fail-on-skip command mode.

10 AI Assistance Disclosure

Generative AI tools assisted with drafting, editing, repository inspection, consistency checks, and implementation. They are not authors. The author remains responsible for the source code, evidence, analyses, figures, tables, citations, limitations, and conclusions.

11 Conclusion

Relaytic-AML demonstrates an artifact-centered approach to agent-assisted financial-crime machine-learning evaluation. The framework binds measurements to source posture, split contracts, feature and leakage policy, model budget, operating points, and test-exposure status, while separate claim gates control what can be released. In the evaluated temporal, graph, external-artifact, and deterministic fixture workflows, Relaytic-AML preserved the tested provenance requirements, produced raw-row-free handoff records, supported recovery from interrupted state, and blocked the injected unsupported claims. These results establish the behavior of the implemented evaluation and release-governance mechanisms, not detector superiority, privacy certification, or production effectiveness. Human and institutional studies remain necessary to assess their effect on expert decisions and operational outcomes.

A Detailed Audit and Reproducibility Records

The appendix keeps concrete audit evidence out of the main reading path while preserving it for reviewers who want to inspect the mechanics. Table 8 records model families and search budgets. The repository stores the corresponding JSON reports with full fields, hashes, and pass criteria.

Table 8: Model families and search budgets.

Track	Families	Features	Search budget	Evidence role
PaySim	tree and boosting candidates, Extra Trees selected	amount, type, time, shifted destination history	14 probes, 5 finalists, seed 42	validation-selected finalist, prior baseline test exposure disclosed
Elliptic	tree/boosting baselines, LightGBM selected	source node features plus same-step graph statistics	16 trials, seed 42	graph-feature evidence row
Elliptic2	LightGBM context row	348 pooled subgraph moments/counts	3 repeated seeds: 11, 42, 73	pinned external-artifact context only

The model-search table records budget shape and evidence role. It is appendix material because it supports auditability without changing the paper’s central architectural claim.

The five comparable PaySim full-training finalists are shown in Table 9. Their validation scores, rather than the small-sample probe scores, determined the competitive winner.

Table 9: Full-training PaySim finalist comparison.

Rank	Finalist	Compact configuration	Validation PR-AUC	Calibration	Test status	Outcome
1	Extra Trees	160 trees, 18 depth, 4 leaf size	0.5687	Platt sigmoid calibration	eligible and evaluated	selected
2=	XGBoost	220 trees, 4 depth, 0.06 learning rate	0.5405	not run	not eligible after validation rank	joint runner-up at displayed precision
2=	Random Forest	120 trees, 18 depth, 5 leaf size	0.5405	not run	not eligible after validation rank	joint runner-up at displayed precision
4	Histogram gradient boosting	160 iterations, 0.08 learning rate, 50 leaf size	0.4674	not run	not eligible after validation rank	lower validation PR-AUC
5	LightGBM	300 trees, 0.03 learning rate, 63 leaves	0.0623	not run	not eligible after validation rank	lower validation PR-AUC

Extra Trees leads the full-training finalist set by 0.0282 PR-AUC over the joint XGBoost and Random Forest runner-up rows at the reported precision. No tie-break was required to select the winner. Nonselected finalists were neither calibrated nor evaluated on test.

Table 10 records the complete operating-point transfer for PaySim and Elliptic.

Table 10: Validation-derived operating-point transfer.

Dataset	Calibration	Threshold	Requested queue	Threshold-selection queue, P/R	Test queue, P/R	Rule
PaySim	Platt sigmoid calibration	0.4045	0.5000%	559/111,601 (0.5009%), 0.5134/0.5053	1,109/123,580 (0.8974%), 0.7033/0.4716	inclusive threshold
Elliptic	Platt sigmoid calibration	0.9998	0.5000%	21/4,145 (0.5066%), 1.0000/0.0792	36/11,184 (0.3219%), 1.0000/0.0566	inclusive threshold

The validation threshold is carried unchanged to test and equality is included. Consequently, the realized test queue may differ from the requested validation fraction.

Table 11 reports the validation surfaces used for model selection, calibration, and threshold selection. The full validation partition selected the model, while the nested chronological sub-windows separated calibration fitting from calibration comparison and threshold choice.

Table 11: Validation surfaces for selection, calibration, and thresholding.

Dataset	Purpose	Boundary	Evaluated units	Positives	Overlap and use
PaySim	Model selection	steps 446-594	228,103	1,552	full validation, contains both nested subsets
PaySim	Calibration fit	steps 446-540	116,502	984	earlier subwindow, disjoint from threshold subset
PaySim	Calibration comparison and threshold	steps 541-594	111,601	568	later subwindow, 0.5% threshold selected here
Elliptic	Model and feature-view selection	time steps 30-39	8,999	1,038	full validation, contains both nested subsets
Elliptic	Calibration fit	time steps 30-35	4,854	773	earlier subwindow, disjoint from threshold subset
Elliptic	Calibration comparison and threshold	time steps 36-39	4,145	265	later subwindow, 0.5% threshold selected here

The calibration and threshold-selection subwindows are disjoint. They are not independent of model selection because the complete validation partition was used to rank finalists or feature views.

The complete serialized **PS-PR** factual cell and its separate claim-gate record are shown below. The JSON retains the stable `missing_evidence` schema field, while the manuscript refers to the record as a missing-evidence record.

Factual evidence cell

```
{
  "cell_type": "metric_evidence_cell",
  "cell_id": "PS-PR",
  "dataset_id": "paysim_temporal_transaction_fraud",
  "split": "temporal test",
  "command": "paysim-competitive --budget-tier competitive",
  "artifact_ref": "paper evidence audit: test_pr_auc",
  "metric": "test_pr_auc",
  "value": 0.638773,
  "budget_tier": "competitive",
  "leakage_posture": "balance fields and raw identifiers excluded",
  "calibration_status": "Platt sigmoid calibration",
  "operating_point_ref": "competitive manifest: selected model test operating point",
  "exposure_status": "fixed_test_previously_exposed",
  "test_exposure_contract": {
    "test_partition_fixed": true,
    "test_partition_previously_exposed": true,
    "competitive_selection_used_test": false,
    "competitive_finalists_tested_after_freeze": 1
  }
}
```

Separate claim-gate record

```
{
  "gate_id": "paysim_p6a_competitive_selected.publication_gate",
  "evidence_cell_ids": [
    "PS-PR"
  ],
  "admissible_use": "bounded PaySim temporal-proxy demonstration",
  "stronger_claim_status": "blocked",
  "gate_reasons": [
    "the fixed test partition had prior P4 reference and P6 baseline exposure",
    "competitive selection did not use test evidence and one finalist was evaluated after protocol freeze"
  ],
  "missing_evidence": [
    "genuinely unseen chronological or external holdout",
    "incumbent queue comparison under the same operating contract"
  ]
}
```

The generated type contract and its authoritative required-field counts are summarized in Table 12.

Table 12: Typed factual evidence contract.

Cell type	Schema	Required fields	Type-specific content	Permitted role
Metric observation	metric-cell v1	17	metric, value, model, calibration, exposure, operating point	detector ranking or review-budget measurement
System invariant	invariant-cell v1	16	invariant name, observed value, state, rowless-export status	factual governance or handoff state, detector metrics prohibited
Disabled-field ablation	metric contract	17	all required metric fields removed	release must block
Missing-field stress	typed metric fixture	14	schema, cell ID, and type retained, remaining fields omitted	schema validation must fail

Metric and invariant records share a factual provenance base, then diverge into type-specific fields. The schema validators reject untyped records, missing required fields, invariant records carrying detector metrics, and factual records containing interpretive claim fields.

The injected risks and observed release behavior can be seen in Table 13.

Table 13: Detailed failure-case fixtures.

Failure mode	Injected risk	Gate/check	Evidence	Expected behavior	Observed result
Leakage-column injection	PaySim balance fields are offered as candidate model inputs.	Leakage feature policy	PS-PR feature policy	Post-event balance fields stay out of allowed features.	4 offered, 4 excluded, 0 used, labels not used as features
Test-set selection violation	A model-selection path tries to use test evidence before the finalist is fixed.	Validation-only selection policy	PS-PR search contract	Only validation evidence may select, calibrate, or threshold the finalist. Prior P4/P6 test exposure must remain disclosed.	validation-only probes, no test selection, one competitive finalist evaluated after protocol freeze
Over-strong claim attempt	Draft wording proposes real-bank superiority or RevClassifyDS parity.	Public claim gate	claim-gate report	Unsupported performance and deployment claims were blocked.	All 6/6 injected unsupported-claim cases were blocked
Rowless handoff redaction	An external-agent packet requests raw rows, private paths, or sensitive fields.	Context export redaction	handoff redaction task	The export contains state and next actions, not raw rows or private paths.	raw rows excluded from export, 8 unsafe fields redacted, 6 blocked fields recorded
Interrupted-run recovery	A user or agent resumes a partial run without knowing which artifact to inspect.	No-lost-user guide	guide recovery task	The guide exposes current state, missing evidence, artifact shortlist, and next actions.	partial run recovered, 8 missing-evidence items recorded, 6 recovery actions exposed

The failure-case fixtures exercise whether the release path refuses leakage features, test-set model selection, over-strong claims, unsafe handoff, and lost-run states. They do not add detector benchmark rows.

Table 14 compares the complete governance path with fixtures in which one control is disabled.

Table 14: Governance machinery ablation.

Path	Disabled machinery	Unsafe signal	Artifact integrity	Handoff / recovery	Interpretation
Full governance path	none	0 unsupported claims, leakage inputs, or raw fields	0 missing fields, 3 table groups	6 recovery actions exposed	Claim gate, leakage policy, redaction, provenance, and recovery guide are all active.
No claim gate	public claim gate	6 unsupported claims	3 table groups unchanged	6 recovery actions exposed	The claim gate is what keeps proxy evidence below unsupported performance, parity, deployment, and business-value claims.
No leakage policy	PaySim feature leakage policy	4 leakage inputs	0 missing fields, 1 unsafe table path	6 recovery actions exposed	The leakage policy prevents post-event simulator fields from becoming apparently strong evidence.
No rowless handoff redaction	external-agent redaction	6 raw fields	3 table groups unchanged	6 raw fields, 6 recovery actions exposed	Rowless handoff reduces the transfer of raw data and local paths to outside agents, but this fixture is not a privacy guarantee.
No evidence-cell required fields	evidence-cell required-field gate	17 missing provenance fields	17 missing fields, 1 unsafe table path	6 recovery actions exposed	Required fields connect a reader-facing measurement to its dataset, split, command, artifact, leakage posture, budget, operating-point provenance, and exposure status. Interpretation remains in a separate gate record.
No interrupted-run recovery guide	no-lost-user guide	0 recovery actions	3 table groups unchanged	0 recovery actions exposed	The recovery guide is what keeps state navigation from depending on repo literacy.

These ablations do not rerun detector training. They change the available governance controls and measure the resulting artifact, handoff, recovery, release, and claim states.

The mechanism, stress signal, and boundary associated with each invariant are collected in Table 15.

Table 15: Governance invariants and evidence map.

Invariant	Mechanism	Evidence and stress signal	Boundary
Evidence-cell provenance	evidence-cell audit plus required-field gate	audit: pass, ablation: 17 required metric fields missing, release blocked	The invariant checks artifact completeness. It does not establish detector optimality.
Claim-strength monotonicity	claim lint, allowed-claims report, publishability matrix, and overclaim failure case	claim lint: pass, overclaim stress test: 6 unsupported claims blocked	The gate is a deterministic release check, it is not an external peer review.
Leakage and selection firewall	feature policy report, split contract, failure fixtures, and leakage ablation	leakage policy: 4 forbidden fields offered, 0 used, leakage stress: 4 leakage fields offered and excluded, 0 used	The current firewall is benchmark-specific, future datasets need their own leakage taxonomy.
Rowless external-agent handoff	handoff evaluator plus redaction failure case	handoff audit: raw rows excluded, 8 unsafe fields redacted, 6 fields blocked, redaction stress: 6 unsafe fields blocked, raw rows excluded	The check evaluates deterministic redaction on fixtures. It is not a broad privacy certification.
Interrupted-run recoverability	no-lost-user guide and partial-run recovery fixtures	recovery audit: partial run recovered, 8 missing items, 6 actions exposed, recovery stress: partial run recovered, 6 actions exposed	The check is deterministic, it does not measure human time-to-recovery.
Benchmark role separation	publishability matrix and allowed-claims report	role matrix: 5 supporting rows allowed, unsupported performance and deployment claims blocked, claim-boundary stress test: 6 public claims blocked	Rows with external or proxy roles cannot be treated as unified leaderboard evidence.
Local-first release safety	release go/no-go, claim lint, and public-claim whitelist	wording lint: pass, claim boundary: unsupported performance claims blocked	Licensed benchmark files are not redistributed, reproduction depends on local access.

The invariant map records release-time rules rather than prose preferences. Each invariant pairs a mechanism with an observed stress signal and an explicit boundary.

The hosted external-score path is illustrated by the rowless fixture in Table 16.

Table 16: Hosted external-score case study.

Component	Observed evidence	Evidence	Admissible interpretation
Adapter input	external-score adapter over a rowless fixture, schema hash 0534b78322c7, content hash 8d3c767fd26d	schema/hash report	The score artifact is described by schema and hash posture, not by raw rows.
Evidence emitted	Required metadata fields present, schema-completeness invariant passed	evidence-cell report	This is a governance check, not detector accuracy, ranking performance, or model novelty.
Rowless handoff	11 exported fields, 16 blocked fields, no raw rows exported	handoff-redaction report	A downstream agent can inspect state without receiving rows, identifiers, paths, or secrets.
Claim gate	hosted detector-output governance only, 5 stronger claims blocked	claim-gate report	The public use is hosted detector-output governance only.

Factual hosted-score cell

```
{
  "cell_type": "invariant_evidence_cell",
  "cell_id": "p19a.external_score.hosted_metadata_completeness",
  "dataset_id": "p19a_hosted_score_fixture",
  "split": "fixture_holdout",
  "command": "relaytic release-safety paper-external-score-proof",
  "artifact_ref": "schema/hash report",
  "artifact_field": "accepted",
  "invariant_name": "hosted_score_metadata_completeness",
  "observed_value": true,
  "invariant_state": "pass",
  "detector_performance_metric": false,
  "operating_point_applicability": "not_applicable",
  "leakage_posture": "rowless_no_training_or_label_data_exported",
  "rowless_export_status": "rowless"
}
```

Separate hosted-score gate

```
{
  "gate_id": "p19a.external_score.hosted_output_gate",
  "evidence_cell_ids": [
    "p19a.external_score.hosted_metadata_completeness"
  ],
  "admissible_use": "hosted detector-output governance only",
  "stronger_claim_status": "blocked",
  "gate_reasons": [
    "the fixture measures hosted-score metadata completeness rather than detector performance",
    "raw rows and entity identifiers are excluded from the export"
  ]
}
```

The hosted-score record is metadata governance only. On the tested fixture, a rowless score artifact is wrapped by a factual schema-and-hash record, a redaction report, and a separate interpretation gate. Its completeness result is not detector accuracy or ranking performance.

Table 17 connects stronger future claims to current admissible uses and to the additional evidence each claim would require.

Table 17: Evidence routing examples.

Stronger future use	Current admissible use	Evidence requirements
Real-bank deployment study	bounded PaySim temporal-proxy demonstration	Partner or bank-approved holdout, incumbent comparison, analyst-review protocol, and legal release gate.
Elliptic2 external-method study	pinned external-artifact context only	Reconstructable upstream artifact provenance, row-level mapping, faithful method execution, and a separately defined comparison protocol.
Graph-native detector release	Elliptic temporal graph-feature evidence path	Graph-native release budget, neural baselines, repeated seeds, and graph-specific ablations.

The blocked-claim rows show how stronger future uses are handled. The gate records current admissible use and the evidence requirements for a stronger interpretation.

Concrete external-agent handoff and interrupted-run recovery records are shown in Table 18.

Table 18: Rowless handoff and interrupted-run recovery examples.

Scenario	Input state	Exported fields	Redacted fields	Observed signal
External-agent handoff	partial run with available guide state	state summary, action options, starter questions, tool contract, artifact shortlist	raw transaction rows, credentials, private paths, raw source files	raw rows excluded from export, 8 unsafe fields redacted, 6 blocked fields recorded
Safe next action	external model asked what to do next	six next actions, six starter questions, command options	unredacted local paths and data rows	6 next actions and 6 starter questions exposed
Interrupted-run recovery	operator returns to partial run without artifact literacy	current state, missing evidence count, canonical artifact shortlist, context-export command	raw benchmark data and private machine paths	partial run recovered, 8 missing-evidence items and 6 actions exposed

The handoff and recovery rows give the practical external-agent story. A second model can receive state, commands, artifacts, and starter questions, while raw rows remain redacted and private paths stay withheld.

Appendix reproduction shortcut:

The full Windows and macOS/Linux regeneration script is kept in the README so the appendix remains readable. The essential local paper path is:

Windows PowerShell:

```
py -3.11 -m pip install -e ".[full]"
py -3.11 -m relaytic.ui.cli release-safety paper-invariants --format json
py -3.11 -m relaytic.ui.cli release-safety paper-release --format json
py -3.11 -m relaytic.ui.cli release-safety paper-release-integrity --candidate --format json
py -3.11 -m relaytic.ui.cli release-safety paper-arxiv-source --format json
py -3.11 -m relaytic.ui.cli release-safety paper-final-preflight --format json
py -3.11 -m pytest -m prepush -q
```

Run `paper-final-preflight` after compiling the arXiv source and copying the PDF into the release bundle. The README includes the exact compile and verification commands.

macOS/Linux:

```
python3 -m pip install -e ".[full]"
python3 -m relaytic.ui.cli release-safety paper-invariants --format json
python3 -m relaytic.ui.cli release-safety paper-release --format json
python3 -m relaytic.ui.cli release-safety paper-release-integrity --candidate --format json
python3 -m relaytic.ui.cli release-safety paper-arxiv-source --format json
python3 -m relaytic.ui.cli release-safety paper-final-preflight --format json
python3 -m pytest -m prepush -q
```

References

Claudio Bellei, Muhua Xu, Ross Phillips, Tom Robinson, Mark Weber, Tim Kaler, Charles E. Leiserson, Arvind, and Jie Chen. The Shape of Money Laundering: Subgraph Representation Learning on the Blockchain with the Elliptic2 Dataset, 2024. URL <https://arxiv.org/abs/2404.19109>.

- Hui Chen, Miao Xiong, Yujie Lu, Wei Han, Ailin Deng, Yufei He, Jiaying Wu, Yibo Li, Yue Liu, and Bryan Hooi. MLR-Bench: Evaluating AI Agents on Open-Ended Machine Learning Research, 2025. URL <https://arxiv.org/abs/2505.19955>.
- Keyang Chen, Mingxuan Jiang, Yongsheng Zhao, Zeping Li, Zaiyuan Chen, Weiqi Luo, Zhixin Li, Sen Liu, Yinan Jing, Guangnan Ye, Xihong Wu, and Hongfeng Chai. TransXion: A High-Fidelity Graph Benchmark for Realistic Anti-Money Laundering, 2026. URL <https://arxiv.org/abs/2604.17420>.
- Tianqi Chen and Carlos Guestrin. XGBoost: A Scalable Tree Boosting System. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 785–794, 2016. doi: 10.1145/2939672.2939785. URL <https://doi.org/10.1145/2939672.2939785>.
- Bruno Deprez, Wei Wei, Wouter Verbeke, Bart Baesens, Kevin Mets, and Tim Verdonck. Advances in Continual Graph Learning for Anti-Money Laundering Systems: A Comprehensive Review. *WIREs Computational Statistics*, 17(3):e70040, 2025. doi: 10.1002/wics.70040. URL <https://doi.org/10.1002/wics.70040>.
- Federal Financial Institutions Examination Council. BSA/AML Examination Manual: Appendix F, Money Laundering and Terrorist Financing Red Flags, 2014. URL <https://bsaaml.ffiec.gov/manual/Appendices/07>. Accessed 2026-07-14.
- Financial Action Task Force. Virtual Assets Red Flag Indicators of Money Laundering and Terrorist Financing, 2020. URL <https://www.fatf-gafi.org/en/publications/Methodsandtrends/Virtual-assets-red-flag-indicators.html>. Accessed 2026-07-14.
- Timnit Gebru, Jamie Morgenstern, Briana Vecchione, Jennifer Wortman Vaughan, Hanna Wallach, Hal Daume III, and Kate Crawford. Datasheets for Datasets. *Communications of the ACM*, 64(12):86–92, 2021. doi: 10.1145/3458723. URL <https://arxiv.org/abs/1803.09010>.
- Pierre Geurts, Damien Ernst, and Louis Wehenkel. Extremely Randomized Trees. *Machine Learning*, 63(1):3–42, 2006. doi: 10.1007/s10994-006-6226-1. URL <https://doi.org/10.1007/s10994-006-6226-1>.
- Mehmet Iscan. Feedback-Normalized Developer Memory for Reinforcement-Learning Coding Agents: A Safety-Gated MCP Architecture, 2026. URL <https://arxiv.org/abs/2605.01567>.
- Maurits Kaptein, Vassilis-Javed Khan, and Andriy Podstavnychy. Runtime Governance for AI Agents: Policies on Paths, 2026. URL <https://arxiv.org/abs/2603.16586>.
- Guolin Ke, Qi Meng, Thomas Finley, Taifeng Wang, Wei Chen, Weidong Ma, Qiwei Ye, and Tie-Yan Liu. LightGBM: A Highly Efficient Gradient Boosting Decision Tree. In *Advances in Neural Information Processing Systems*, volume 30, 2017. URL https://proceedings.neurips.cc/paper_files/paper/2017/hash/6449f44a102fde848669bdd9eb6b76fa-Abstract.html.
- Martin Kleppmann, Adam Wiggins, Peter van Hardenberg, and Mark McGranaghan. Local-First Software: You Own Your Data, in Spite of the Cloud. In *Proceedings of the 2019 ACM SIGPLAN International Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software*, pages 154–178, 2019. doi: 10.1145/3359591.3359737. URL <https://doi.org/10.1145/3359591.3359737>.

- Edgar Alonso Lopez-Rojas, Ahmad Elmir, and Stefan Axelsson. PaySim: A Financial Mobile Money Simulator for Fraud Detection. In *Proceedings of the 28th European Modeling and Simulation Symposium*, 2016. URL https://www.msc-les.org/proceedings/emss/2016/E MSS2016_249.pdf.
- Margaret Mitchell, Simone Wu, Andrew Zaldivar, Parker Barnes, Lucy Vasserman, Ben Hutchinson, Elena Spitzer, Inioluwa Deborah Raji, and Timnit Gebru. Model Cards for Model Reporting. In *Proceedings of the Conference on Fairness, Accountability, and Transparency*, pages 220–229, 2019. doi: 10.1145/3287560.3287596. URL <https://arxiv.org/abs/1810.03993>.
- Prathamesh Vasudeo Naik, Naresh Kumar Dintakurthi, Zhanghao Hu, Yue Wang, and Robby Qiu. Co-Investigator AI: The Rise of Agentic AI for Smarter, Trustworthy AML Compliance Narratives, 2025. URL <https://arxiv.org/abs/2509.08380>.
- Prathamesh Vasudeo Naik, Naresh Dintakurthi, and Yue Wang. Rethinking LLMOps for Fraud and AML: Building a Compliance-Grade LLM Serving Stack, 2026. URL <https://arxiv.org/abs/2605.11232>.
- Helen Pervez, Suyash Gaurav, Jukka Heikkonen, and Jatin Chaudhary. Governance-as-a-Service: A Multi-Agent Framework for AI System Compliance and Policy Enforcement, 2025. URL <https://arxiv.org/abs/2508.18765>.
- Joelle Pineau, Philippe Vincent-Lamarre, Koustuv Sinha, Vincent Lariviere, Alina Beygelzimer, Florence d’Alche Buc, Emily Fox, and Hugo Larochelle. Improving Reproducibility in Machine Learning Research. *Journal of Machine Learning Research*, 22(164):1–20, 2021. URL <https://www.jmlr.org/papers/v22/20-303.html>.
- Erfan Pirmorad. Exploring the In-Context Learning Capabilities of LLMs for Money Laundering Detection in Financial Graphs, 2025. URL <https://arxiv.org/abs/2507.14785>.
- John C. Platt. Probabilistic Outputs for Support Vector Machines and Comparisons to Regularized Likelihood Methods. In *Advances in Large Margin Classifiers*, pages 61–74. MIT Press, 1999.
- Chung-Hoo Poon, James Tin Yau Kwok, Calvin Chow, and Jang-Hyeon Choi. LineMVGNN: Anti-Money Laundering with Line-Graph-Assisted Multi-View Graph Neural Networks. *AI*, 6(4):69, 2025. doi: 10.3390/ai6040069. URL <https://doi.org/10.3390/ai6040069>.
- Takaya Saito and Marc Rehmsmeier. The Precision-Recall Plot Is More Informative than the ROC Plot When Evaluating Binary Classifiers on Imbalanced Datasets. *PLOS ONE*, 10(3): e0118432, 2015. doi: 10.1371/journal.pone.0118432. URL <https://doi.org/10.1371/journal.pone.0118432>.
- Kiwhan Song, Mohamed Ali Dhraief, Muhua Xu, Locke Cai, Xuhao Chen, Arvind, and Jie Chen. Identifying Money Laundering Subgraphs on the Blockchain. In *Proceedings of the 5th ACM International Conference on AI in Finance*, 2024. doi: 10.1145/3677052.3698635. URL <https://arxiv.org/abs/2410.08394>.
- Giulio Starace, Oliver Jaffe, Dane Sherburn, James Aung, Jun Shern Chan, Leon Maksin, Rachel Dias, Evan Mays, Benjamin Kinsella, Wyatt Thompson, Johannes Heidecke, Amelia Glaese, and Tejal Patwardhan. PaperBench: Evaluating AI’s Ability to Replicate AI Research, 2025. URL <https://arxiv.org/abs/2504.01848>.
- Haseeb Tariq and Marwan Hassani. Extracting Money Laundering Transactions from Quasi-Temporal Graph Representation, 2026. URL <https://arxiv.org/abs/2604.02899>.

- Mark Weber, Giacomo Domeniconi, Jie Chen, Daniel Karl I. Weidele, Claudio Bellei, Tom Robinson, and Charles E. Leiserson. Anti-Money Laundering in Bitcoin: Experimenting with Graph Convolutional Networks for Financial Forensics, 2019. URL <https://arxiv.org/abs/1908.02591>.
- Hjalmar Wijk, Tao Roa Lin, Joel Becker, Sami Jawhar, Neev Parikh, Thomas Broadley, Lawrence Chan, Michael Chen, Joshua M. Clymer, Jai Dhyani, Elena Elicheva, Katharyn Garcia, Brian Goodrich, Nikola Jurkovic, Megan Kinniment, Aron Lajko, Seraphina Nix, Lucas Jun Koba Sato, William Saunders, Maksym Taran, Ben West, and Elizabeth Barnes. RE-Bench: Evaluating Frontier AI R&D Capabilities of Language Model Agents against Human Experts. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 66772–66832, 2025. URL <https://proceedings.mlr.press/v267/wijk25a.html>.
- Yihan Xia and Taotao Wang. ResearchLoop: An Evidence-Gated Control Plane for AI-Assisted Research, 2026. URL <https://arxiv.org/abs/2605.28282>.
- Hang Xu, Ling Yue, Chaoqian Ouyang, Libin Zheng, Shaowu Pan, Shimin Di, and Min-Ling Zhang. FactReview: Evidence-Grounded Reviews with Literature Positioning and Execution-Based Claim Verification, 2026. URL <https://arxiv.org/abs/2604.04074v1>. Version 1.
- Yifan Yang, Ziyang Gong, Weiquan Huang, Qihao Yang, Ziwei Zhou, Zisu Huang, Yan Li, Xuemei Gao, Qi Dai, Bei Liu, Kai Qiu, Yuqing Yang, Dongdong Chen, Xue Yang, and Chong Luo. SkillOpt: Executive Strategy for Self-Evolving Agent Skills, 2026. URL <https://arxiv.org/abs/2605.23904>.
- Haojie Ye, Arjun Laxman, Yichao Yuan, Krisztian Flautner, and Nishil Talati. BlazingAML: High-Throughput Anti-Money Laundering (AML) via Multi-Stage Graph Mining, 2026. URL <https://arxiv.org/abs/2604.12241>.
- Matei Zaharia, Andrew Chen, Aaron Davidson, Ali Ghodsi, Sue Ann Hong, Andy Konwinski, Siddharth Murching, Tomas Nykodym, Paul Ogilvie, Mani Parkhe, Fen Xie, and Corey Zumar. Accelerating the Machine Learning Lifecycle with MLflow. *IEEE Data Engineering Bulletin*, 41(4):39–45, 2018. URL https://people.eecs.berkeley.edu/~matei/papers/2018/ieee_mlflow.pdf.